HPC FINAL LAB PROJECT · P08

Live Demo Script

Exactly what to run, what will appear on screen, and what to say — from switching on the laptop to the closing sentence. Keep this open next to you during the presentation.

THE ONE COMMAND

```
powershell -ExecutionPolicy Bypass -File scripts\demo.ps1
```

Run it in PowerShell from the project folder, with Docker Desktop running. Press ENTER at each step.



≈ 6 min	9	3	1
talk + demo	guided demo steps	graphs to show	closing sentence to remember

1 Before you start

TONIGHT — PRACTICE RUN (ONCE)

- [ ] Start **Docker Desktop** (Start menu) → wait for **Engine running**.
- [ ] Open the folder `HPCG-Performance-Analyzer` in File Explorer, type `powershell` in the address bar, press Enter.
- [ ] Run the demo command (below) and press ENTER through all 9 steps.
- [ ] Say the **SAY** lines out loud while it runs — twice.
- [ ] Run `start results\dashboard.html` and practise the 3 graphs (Section 5).
- [ ] Read the "Numbers to memorise" page of the handbook.

TOMORROW — 10 MINUTES BEFORE

- [ ] Laptop on **charger** (battery mode slows the CPU).
- [ ] Close Chrome tabs, games, video calls.
- [ ] Start Docker Desktop → **Engine running**.
- [ ] PowerShell open in the project folder.
- [ ] `results\dashboard.html` open in the browser.
- [ ] GitHub repo page open in another tab.
- [ ] This script and the handbook open/printed.

Commands you need

Purpose	Command (PowerShell, in the project folder)
Guided demo (main one)	`powershell -ExecutionPolicy Bypass -File scripts\demo.ps1`
Same, without pauses	`powershell -ExecutionPolicy Bypass -File scripts\demo.ps1 -NoPause`
Linux shell for your own commands	`powershell -ExecutionPolicy Bypass -File scripts\demo.ps1 -Shell`
Open results dashboard	`start results\dashboard.html`
Open the report	`notepad report\project_report.md`

Where is the project?

`C:\Users\anutt\Downloads\HPC_Final_Lab_Project\HPCG-Performance-Analyzer`

On another PC: install Docker Desktop, clone `github.com/Anuttaman216/HPC_FInal_Lab_Project` , run the same command (first run downloads and builds the toolchain, ≈ 5 min, needs internet).

2 Session flow

1 Problem 0:30	2 What we built 0:45	3 Live demo 3:00	4 Results 1:00	5 Close 0:30
why HPCG	4 versions	scripts\demo.ps1	3 graphs	one sentence

Total ≈ 6 minutes, then viva questions

3 Opening — say this (≈ 1 min 15 s)

SAY **Problem.** "Supercomputers are ranked in the TOP500 list using a benchmark called HPL. HPL solves a dense system of equations, so it mostly measures how fast the processor does arithmetic. But real scientific programs — weather, fluid flow, engineering — solve *sparse* systems, and they are slowed down by moving data from memory, not by arithmetic. That is why Dongarra and Heroux proposed HPCG, the High Performance Conjugate Gradient benchmark, in 2013."

SAY **What we built.** "Our project is an HPCG-inspired, educational benchmark. It builds a sparse matrix with about 2 million equations from a 3D grid, and solves it with the Conjugate Gradient method in four ways: sequential on one core, OpenMP with threads sharing memory, MPI with separate processes that send messages — using two different communication methods — and a hybrid of MPI and OpenMP. It has 10 automated tests, and it measures time, speedup, efficiency and communication, and draws the graphs automatically. Let me show it running."

4 The live demo — step by step

Run the demo command. Each step prints a short explanation, then waits — say the line, then press ENTER. Live timings will differ slightly from the examples below; that is normal.

0 Build check

5 s

Confirms the program is compiled.

```
build/hpcg_benchmark already built - skipping.
```

SAY "The code is C++17, compiled with CMake and -O3 optimisation, with OpenMP and MPI. It runs inside a Linux Docker container, so anyone can reproduce it exactly."

1 The machine

15 s

Prints compiler, library versions, CPUs and memory.

```
(press ENTER) build/hpcg_benchmark --info
```

```
compiler          : g++ 13.3.0
build flags       : -O3 -DNDEBUG
OpenMP            : 4.5 (201511)
MPI library       : Open MPI v4.1.6
logical CPUs      : 16
memory total      : 11.50 GiB
```

SAY "This is our environment: g++ 13, OpenMP 4.5, Open MPI 4.1, 16 logical CPUs. It is recorded with every result so the experiment is reproducible."

2 Correctness tests

20 s

Runs all 10 automated tests (CTest).

```
(press ENTER) ctest
```

```
10/10 Test #10: e2e_mpi_halo .....................   Passed
100% tests passed, 0 tests failed out of 10
```

SAY "Before measuring speed we prove correctness. These 10 tests check that the matrix is symmetric positive definite, that CG finds the exact answer, and that OpenMP, MPI and hybrid give the same answer as the sequential version. MPI is tested with 1, 2, 3 and 4 processes — 3 gives an uneven split of rows."

POINT Point at **100% tests passed**.

3 Sequential baseline

20 s

One core solves a 64×64×64 problem (262,144 equations).

```
(press ENTER) build/hpcg_benchmark --mode sequential --grid 64 --reps 3
```

```
RESULT mode=sequential ... rows=262144 nnz=1810432 procs=1 threads=1 iters=158
  status=converged relres=9.032e-09 true_relres=9.032e-09 max_err=2.958e-08
  t_median=0.59s ... GFLOPS=1.67
```

SAY "One core solves 262 thousand equations. It converged in 158 iterations, and the error from the exact answer is about 3×10^{-8}. This is our baseline."

POINT Point at **iters=158**, **status=converged**, **max_err**, **t_median**.

4 OpenMP — 4 threads

20 s

Same problem, 4 threads sharing memory.

```
(press ENTER) OMP_NUM_THREADS=4 build/hpcg_benchmark --mode openmp --grid 64 --reps 3
```

```
RESULT mode=openmp ... procs=1 threads=4 iters=158 status=converged
  max_err=2.958e-08 t_median=0.25s ... GFLOPS=4.02
```

SAY "Now 4 threads sharing memory. Same 158 iterations, same accuracy — so it is exactly the same computation — but faster. Each loop is split across threads, and the dot products use OpenMP reductions, so there are no race conditions."

POINT Compare **t_median** with step 3.

5a MPI — Allgatherv

25 s

4 separate processes; every process receives the whole vector each iteration.

```
(press ENTER) mpirun -np 4 build/hpcg_benchmark --mode mpi --comm allgather --grid 64 --reps 3
```

```
  rank 0 receives 196608 doubles per exchange via MPI_Allgatherv
RESULT mode=mpi comm=allgather procs=4 threads=1 iters=158 status=converged
  t_median=0.38s comm_frac=0.323
```

SAY "Now distributed memory: 4 separate processes, each owning a quarter of the rows. With Allgatherv every process receives the *whole* vector every iteration — about 196 thousand numbers. Around a third of the time is communication."

POINT Point at **receives 196608 doubles** and **comm_frac**.

5b MPI — halo exchange

25 s

Same, but each process receives only its neighbours' boundary values.

```
(press ENTER) mpirun -np 4 build/hpcg_benchmark --mode mpi --comm halo --grid 64 --reps 3
```

```
  rank 0 receives 4096 doubles per exchange from 1 halo neighbour(s)
RESULT mode=mpi comm=halo procs=4 threads=1 iters=158 status=converged
  t_median=0.28s comm_frac=0.166
```

SAY "With halo exchange each process gets only the 4,096 boundary values it actually needs — 48 times less data — so communication drops and the run is faster. How you communicate matters more than how many processes you use."

POINT Compare **4096** vs **196608**, and the two **comm_frac** values.

6 Hybrid MPI + OpenMP

20 s

2 MPI processes × 2 OpenMP threads each.

```
(press ENTER) OMP_NUM_THREADS=2 mpirun -np 2 build/hpcg_benchmark --mode hybrid --comm halo --grid 64 --reps 3
```

```
RESULT mode=hybrid comm=halo procs=2 threads=2 iters=158 status=converged
```

SAY "Hybrid: MPI between processes and OpenMP threads inside each. Only the main thread calls MPI — that is the FUNNELED thread level, which the program checks at start-up. This is how real clusters are programmed."

7 Memory bandwidth

15 s

STREAM-style triad: how many GB/s memory delivers with 1, 4, 8 threads.

```
(press ENTER) build/stream_triad --threads 1,4,8 --mb 64 --reps 5
```

```
threads,array_mb,best_GBps,median_GBps
1,64,...
4,64,...
8,64,...
```

SAY "This explains everything. Memory bandwidth grows much less than the number of cores. CG does only about 0.13 operations per byte it reads, so it is limited by memory speed, not by the processor. That is exactly what HPCG was designed to expose."

POINT In our full measurement: **21.9 GB/s** (1 thread) → **28.9 GB/s** (8 threads) = only **1.32×**.

8 Full saved results

30 s

Prints the summary of the complete 50-configuration campaign (128^3 grid).

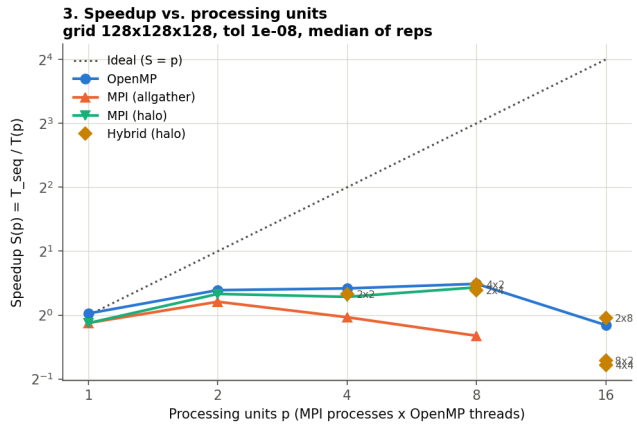
```
(press ENTER) sed -n '3,25p' results/summary.md
```

```
| Sequential       | 1 | 296 | 5.9527 | 1.00 | ...
| OpenMP 8t        | 8 | 296 | 4.2401 | 1.40 | ...
| MPI-allgather 8p | 8 | 296 | 7.4464 | 0.80 | ... 0.506
| MPI-halo 8p      | 8 | 296 | 4.4047 | 1.35 | ... 0.115
```

SAY "The full experiment ran 50 configurations on 2.1 million equations; every one needed exactly 296 iterations. The best speedup is 1.40 times with 8 threads — matching the 1.32 times growth in memory bandwidth. With 8 processes Allgatherv is even slower than one core, while halo exchange is 1.69 times faster than Allgatherv."

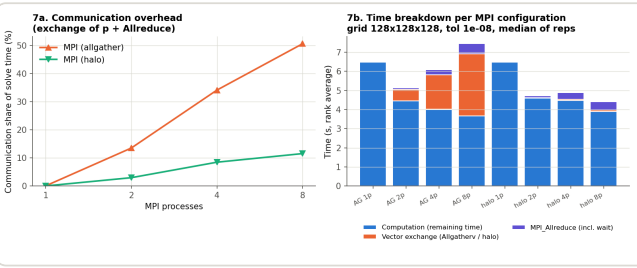
5 Show the graphs (≈ 1 min)

Switch to the browser with `results\dashboard.html` (or open the PNGs in `results\graphs\`).



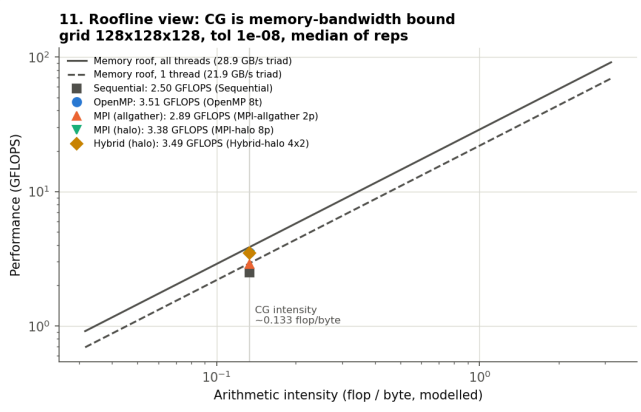
SAY "This is speedup against the number of cores. The dotted line is the ideal. All our versions flatten at about 1.4 — that is the **memory wall**. Using 16 threads is even slower than 8, because the extra hyper-threads add no memory bandwidth."

POINT The flat lines near $2^{0.5} \approx 1.4$, and the drop at 16.



SAY "Communication share for MPI. Orange is Allgatherv — it grows to 51% at 8 processes. Green is halo exchange — it stays around 11%. Same problem, same answer; only the communication design changes."

POINT Orange line rising steeply vs green line staying low.



SAY "The roofline model. CG does 0.13 operations per byte, so the maximum possible speed is set by the memory-bandwidth line: about 3.8 GFLOPS. Our points sit just below it — about 90% of the hardware limit. So the code is efficient; the memory is the bottleneck."

POINT The points touching the sloped line.

6 Closing — say this (≈ 30 s)

SAY "To conclude: we built and tested an HPCG-inspired Conjugate Gradient benchmark in four parallel versions. Our measurements show that for sparse problems, adding cores only helps until memory bandwidth runs out, and that the communication method matters more than the number of processes. In one sentence: **for sparse scientific codes, bytes per second matter more than flops per second — which is why HPCG complements HPL for ranking supercomputers.** Thank you — I'm happy to take questions."

7 If the examiner asks you to run something

Open the Linux shell, then type the command. Keep `--grid` at 128 or below so runs finish in seconds. Type `exit` to leave.

```
PS> powershell -ExecutionPolicy Bypass -File scripts\demo.ps1 -Shell
```

They ask…	Type this (inside the shell)
"Show the options"	`./build/hpcg_benchmark --help`
"Run with 8 threads"	`OMP_NUM_THREADS=8 ./build/hpcg_benchmark --mode openmp --grid 64`
"Run MPI with 2 / 3 processes"	`mpirun -np 3 ./build/hpcg_benchmark --mode mpi --comm halo --grid 64`
"Run a bigger problem"	`./build/hpcg_benchmark --mode sequential --grid 100`
"Change the tolerance"	`./build/hpcg_benchmark --mode sequential --grid 64 --tol 1e-4` (fewer iterations)
"Run exactly 50 iterations"	`./build/hpcg_benchmark --mode sequential --grid 64 --tol 0 --max-iters 50`
"Run the tests again"	`cd build && ctest --output-on-failure && cd ..`
"What if the problem is too big?"	`./build/hpcg_benchmark --grid 2000` → the memory guard refuses safely
"Show the raw data"	`head results/benchmark.csv` or `cat results/summary.md`

Reading the RESULT line

Field	Meaning
`iters`	Number of CG iterations needed
`status`	`converged` = reached the tolerance (other values: max_iterations, breakdown, non_finite)
`relres` / `true_relres`	Remaining error ‖r‖/‖b‖ — tracked during CG / recomputed at the end (they should match)
`max_err`	Largest distance from the exact answer (all ones)
`t_median`	Median solve time over the repetitions

`comm_frac`	Share of time spent in MPI communication
`GFLOPS`	Billions of floating-point operations per second (from the operation-count model)

8 If something goes wrong

What you see	What to do
"Docker Desktop is not running"	Start Docker Desktop, wait for **Engine running** (30–60 s), run the command again.
"running scripts is disabled"	Use the full command with `-ExecutionPolicy Bypass` as written above.
"the input device is not a TTY"	You are in Git Bash — use **PowerShell**, or add `-NoPause`.
"not enough slots" from mpirun	Use fewer processes, e.g. `-np 2`.
Hybrid run very slow	Add `--bind-to none` after `mpirun`.
Times worse than the report	Plug in the charger, close apps. Say: "timings vary run to run, that's why we report the median of repetitions."
Nothing works at all	Show the saved real results: `results\dashboard.html`, `results\summary.md`, `results\logs\`, `results\test_results.txt`.

9 Quick reference card

2.1 M equations (128^3 grid)	**296** iterations, all versions	**5.95 → 4.24 s** sequential → OpenMP 8t	**1.40×** best speedup
1.32× bandwidth growth 1→8	**0.13** flops per byte	**51% / 11.5%** comm: Allgatherv / halo	**1.69×** halo vs Allgatherv

The 4 versions in one line each

- **Sequential** — one core, the baseline.
- **OpenMP** — threads share memory; `parallel for` + reductions.
- **MPI** — processes with own memory; rows split; Allgatherv or halo; Allreduce for dot products.
- **Hybrid** — MPI processes with OpenMP threads; FUNNELED.

CG in one breath

Guess x, error r = b − Ax, direction p = r. Repeat: $\alpha = r{\cdot}r \,/\, p{\cdot}Ap \rightarrow x \mathrel{+}= \alpha p \rightarrow r \mathrel{-}= \alpha Ap \rightarrow$ stop if $\|r\|/\|b\| < 10^{-8} \rightarrow \beta = r'{\cdot}r' \,/\, r{\cdot}r \rightarrow p = r + \beta p$. Each iteration: 1 sparse matrix-vector product, 2 dot products, 3 vector updates.

Why only 1.4× on 8 cores

CG moves ~7.5 bytes per operation. Memory bandwidth only grows 1.32× from 1 to 8 cores, so the cores wait for memory. We reach ~90% of the measured memory roof — the hardware is the limit, not the code.

Honesty line

"This is HPCG-*inspired* and educational — not the official HPCG. Official HPCG adds a multigrid preconditioner, a 27-point stencil and strict run rules. Our numbers are not HPCG or TOP500 results."

The closing sentence

For sparse scientific codes, **bytes per second matter more than flops per second** — which is why HPCG complements HPL.